TP - Expressions regulières

1 Outils du Shell GNU

1 - Une Debian classique fournit deux outils de recherche de texte, `grep` et `egrep`, ce dernier utilisant par défaut les Regex "ERE". A l'aide de la commande `which`, donner la commande qui va donner la localisation de ces deux outils, et le résultat :

Commande : which egrep grep

localisation : `grep` : /usr/bin/grep `egrep` : /usr/bin/egrep

2 - En utilisant une substitution de commande utilisant la commande `file`, donner la commande qui va donner la nature de ces deux fichiers, ainsi que le résultat :

Commande : file $(which grep egrep)

type : `grep` : /bin/grep: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked . `egrep` : /bin/egrep: POSIX shell script text executable

3 - En utilisant une substitution de commande utilisant `du --bytes`, donner la commande qui va donner la taille de ces deux fichiers, ainsi que leur taille :

Commande : echo $(wc --bytes $(which grep egrep))

taille : `grep` : 203152 /usr/bin/grep 41 total `egrep` : /usr/bin/egrep 203193 total

4 - Pour ce dernier, l'afficher avec `cat`. Que pouvez-vous dire ? egrep ne sert qu'a appeller la commande grep

2 Mise en pratique

Pour les exercices suivants, vous pourrez faire des essais préliminaires avec les outils en ligne donnés dans le cours, mais on demande une validation avec `grep` avec l'option `-P` (utilisation du dialecte "PCRE").

2.1 Noms de fichiers

Proposer une Regex qui va permettre de détecter des noms de fichiers de type image, à la condition qu'ils respectent les critères habituels : extension séparée du nom par un point, pas de fichier caché (commençant par un point)

Vous validerez votre expression avec ce fichier test :
`https://github.com/skramm/but3_rt/blob/main/ressources_TP/regex/noms_fichiers.txt`

```
img0912.jpg
updated_img0912.png
favicon.gif
documentation.html
.bash_profile
img0912.jpg.tmp
access.lock
workspace.doc
.a.b.jpg
azertyjpg
```

grep -P "^[^\.].*\.(jpg|png|gif)$" noms_fichiers.txt

2.2 Nombres

Proposer une regex qui va matcher sur les chaines suivantes :

`3,14529 -255,34 128 1,9e10 123.340,00`

grep -P "^\-?+\d+[,|.]?e?\d*[,|.]?\d+$" nombres.txt

en évitant de matcher sur les suivantes :
`720p 384$ 248.22€`

Vous utiliserez ce fichier pour la validation :
`https://github.com/skramm/but3_rt/blob/main/ressources_TP/regex/nombres.txt`

2.3 Numéros de téléphone (Américains)

Dans un champ de texte, les utilisateurs doivent saisir des numéros de téléphone américains. Par exemple :
`415-555-1234` `650-555-2345` `(416)555-3456` `202 555 4567` `4035555678` `1 416 555 9292`

Vous utiliserez ce fichier pour la validation :
`https://github.com/skramm/but3_rt/blob/main/ressources_TP/regex/no_tel_USA.txt`

2.4 Adresses IPv4

Matcher une adresse IP n'est pas si simple qu'il n'y parait : Il s'agit de 3 groupes de 3 chiffres, mais qui doivent respecter la contrainte de ne pas dépasser 255.

Commencer par écrire une Regex qui matche sur une valeur entre 1 et 255 avec un point derrière. Vous utiliserez ce fichier pour la validation :
`https://github.com/skramm/but3_rt/blob/main/ressources_TP/regex/adresses_ipv4_1.txt`

Puis l'étendre pour avoir la regex complète :

Vous utiliserez ce fichier pour la validation :
`https://github.com/skramm/but3_rt/blob/main/ressources_TP/regex/adresses_ipv4_2.txt`

3 Analyse d'un fichier système

Le fichier `/etc/services` liste les services réseau et le port qui leur est assigné. Examiner ce fichier (**sans le modifier ! ! !**) et donner les commandes suivantes :

1 - Analyser la construction de ce fichier. En particulier, essayer l'outil `xxd` pour voir le/les caractère(s) utilisé(s) pour séparer les colonnes.

2 - Donner le nombre de services utilisant un port TCP :
→ Nbe=

3 - En prenant à la place du fichier réel le fichier ici :
`https://github.com/skramm/but3_rt/blob/main/ressources_TP/regex/services.txt`
Donner le nombre de services utilisant un port UDP et avec un numéro de port inférieur à 1023.
→ Nbe=